SBB-Verspätungsanalyse – Dokumentation (M323)

Projektgruppe: Aimen Labidi, Elias · Version 2.0

1. Projektantrag

Projekttitel	SBB-Verspätungsanalyse
Ausgangssituation	Pendler und Planer wissen oft nicht, an welchen Bahnhöfen und bei welchen Zugkategorien Verspätungen und Ausfälle am häufigsten sind. Die Applikation wertet die offiziellen Ist-Daten der SBB aus und zeigt die Pünktlichkeit pro Bahnhof, Ausfälle pro Tag und die grössten Verspätungen.
Datengrundlage	SBB Ist-Daten (opentransportdata.swiss), reduziert auf 7 Tage (03.–09.11.2025), 5 Bahnhöfe (Basel SBB, Bern, Liestal, Olten, Zürich HB) und Züge der Kategorien S, RE, IR, IC: 26 984 Halte in `data/ist_daten.csv`. **1. Dimension:** Betriebstage · **2. Dimension:** Bahnhöfe pro Tag · **3. Dimension:** Halte pro Bahnhof (Linie, Kategorie, Soll-, Ist-Zeit, Ausfall) `[("03.11.2025", [("Basel SBB", [{linie, kategorie, soll, ist, ausfall}, ...]), ...]), ...]`
Produktfunktionen	1. **Filter:** nur gefahrene Halte mit Ist-Zeit 2. **Map:** Verspätung in Minuten berechnen (zu früh = 0), pünktlich = unter 3 Min. 3. **Reduce:** Ø Verspätung und Pünktlichkeitsquote pro Bahnhof 4. **Reduce:** Anzahl Zugausfälle pro Tag (plus Total) 5. **Filter + Reduce:** grösste Verspätung pro Zugkategorie 6. **Sortieren:** Ranking der Bahnhöfe nach Pünktlichkeit
Technologien	Python 3.12+, nur Standardbibliothek · Konsolenapplikation · funktionale Elemente: `filter`, `map`, `functools.reduce`, Lambdas, `itertools.groupby`, Comprehensions
Output	Formatierter Text auf der Konsole, siehe Kapitel 3

Beide Programmversionen lesen **dieselbe Datei** `data/ist_daten.csv` mit derselben Funktion `lade_daten` (V1 mit Schleifen, V2 mit `groupby`) und arbeiten auf derselben 3D-Struktur.

2. Gewählte Programmiersprache: Python

Python ist eine imperative, objektorientierte Sprache. Sie bietet aber viele funktionale Sprachelemente, mit denen man ohne Schleifen und ohne Veränderung von Daten programmieren kann.

Warum Python? Wir kennen Python aus anderen Modulen, das Einlesen von CSV-Dateien und das Rechnen mit Datum und Zeit sind in der Standardbibliothek enthalten (`csv`, `datetime`), und es braucht keine Kompilierung. Weil Python beide Stile unterstützt, lassen sich V1 (imperativ) und V2 (funktional) in derselben Sprache direkt vergleichen.

Funktionen als Werte (First-Class-Functions). Funktionen sind Objekte. Man kann sie in Variablen speichern und anderen Funktionen übergeben. In V2 wird `gefahren` an `filter` und `mit_verspaetung` an `map` übergeben:

```
halte = list(map(mit_verspaetung, filter(gefahren, alle_halte(tage))))
```

Funktionen höherer Ordnung: filter, map, reduce. `filter(f, daten)` behält die Elemente, für die `f` `True` liefert. `map(f, daten)` wendet `f` auf jedes Element an. `functools.reduce(f, daten, start)` fasst alle Elemente zu einem Wert zusammen. `filter` und `map` liefern Iteratoren, die erst bei Bedarf rechnen (Lazy Evaluation).

```
reduce(lambda anzahl, halt: anzahl + halt["ausfall"], halte, 0)   # Anzahl Ausfälle
```

Lambda-Ausdrücke. Anonyme Funktionen für kurze Ausdrücke, z. B. als Sortierschlüssel. Ein Lambda darf nur **einen Ausdruck** enthalten, keine Anweisungen. Für mehr Logik schreibt man eine normale Funktion mit `def`.

```
sorted(statistiken, key=lambda stat: (-stat[3], stat[2], stat[0]))
```

Comprehensions und Generatoren. `[f(x) for x in daten if bedingung]` verbindet Map und Filter in einer Zeile. Mit runden Klammern entsteht ein Generator, der die Werte erst bei Bedarf erzeugt.

```
[{**halt, "tag": tag, "bahnhof": bahnhof} for tag, bahnhoefe in tage
 for bahnhof, halte in bahnhoefe for halt in halte]
```

Unveränderlichkeit (Immutability). Tupel und Strings sind unveränderlich. Listen und Dicts sind veränderlich, deshalb erzeugt V2 statt einer Änderung immer ein neues Objekt: `{**halt, "verspaetung": minuten}`.

Pure Functions. Alle Auswertungsfunktionen in V2 hängen nur von ihren Parametern ab und haben keine Seiteneffekte. Nur `lade_daten` (Datei lesen) und `main` (Ausgabe) arbeiten mit der Aussenwelt.

Modul itertools. `groupby` gruppiert aufeinanderfolgende Elemente mit gleichem Schlüssel. Damit baut V2 die 3D-Struktur ohne Schleifen mit `append` auf.

Grenzen von Python. Es gibt keine Tail-Call-Optimierung (tiefe Rekursion führt zu `RecursionError`), Lambdas sind auf einen Ausdruck beschränkt, und Unveränderlichkeit wird nicht erzwungen, sondern ist eine Disziplin des Programmierers.

3. Output

`python v1/main.py` und `python v2/main.py` erzeugen exakt denselben Output (geprüft mit `tests/test_gleicher_output.py`):

```
============================================================
SBB-Verspätungsanalyse 03.11.2025 – 09.11.2025
Pünktlich = weniger als 3 Minuten Verspätung
============================================================

1) Statistik pro Bahnhof
Bahnhof        Halte   Ø Verspätung   Pünktlich
Basel SBB       2226        1.0 min       90.3%
Bern            7008        0.7 min       95.9%
Liestal         1329        1.1 min       90.9%
Olten           4128        0.8 min       94.4%
Zürich HB      11666        1.1 min       92.4%

2) Zugausfälle pro Tag
03.11.2025     69
04.11.2025     39
05.11.2025     35
06.11.2025     47
07.11.2025     46
08.11.2025     67
09.11.2025     71
Total         374

3) Grösste Verspätung pro Kategorie
S     28.7 min  S12    Zürich HB    03.11.2025
RE    17.0 min  RE24   Olten        04.11.2025
IR    63.0 min  IR37   Zürich HB    03.11.2025
IC    58.5 min  IC6    Olten        06.11.2025

4) Ranking Pünktlichkeit
1. Bern          95.9%  (Ø 0.7 min)
2. Olten         94.4%  (Ø 0.8 min)
3. Zürich HB     92.4%  (Ø 1.1 min)
4. Liestal       90.9%  (Ø 1.1 min)
5. Basel SBB     90.3%  (Ø 1.0 min)
```

4. Fazit

Vergleich am Beispiel von Funktion 4 (Ausfälle pro Tag):

V1 (imperativ):

```
resultat = []
for tag, bahnhoefe in tage:
    anzahl = 0
    for _, halte in bahnhoefe:
        for halt in halte:
            if halt["ausfall"]:
                anzahl += 1
    resultat.append((tag, anzahl))
```

V2 (funktional):

```
[(tag, reduce(lambda anzahl, halt: anzahl + halt["ausfall"],
              (halt for _, halte in bahnhoefe for halt in halte), 0))
 for tag, bahnhoefe in tage]
```

Nutzen für unser Projekt und Unterschied zur imperativen Lösung. In V1 braucht jede Auswertung verschachtelte Schleifen, Hilfsvariablen und Zähler, die laufend verändert werden (z. B. `anzahl += 1`, `werte[0] += 1`). In V2 beschreibt der Code, was berechnet wird: `filter(gefahren, …)` → `map(mit_verspaetung, …)` → `reduce(…)`. Jede Funktion ist klein, pur und einzeln testbar, und weil keine Daten verändert werden, kann eine Funktion keine andere durch Seiteneffekte stören. Der Nachteil: Ein `reduce` mit Lambda (z. B. in `statistik_pro_bahnhof`) ist für Einsteiger schwerer zu lesen als eine Schleife.

Erneuter Einsatz. Für Datenauswertungen (Filtern, Umrechnen, Gruppieren, Zusammenfassen) würden wir wieder funktional arbeiten, vor allem Comprehensions, `map`/`filter` und `sorted` mit `key`. Für sehr komplexe Akkumulationen würden wir lesbare Hilfsfunktionen statt verschachtelter Lambdas schreiben.

Anwendungsfälle im Betrieb. Auswerten von Log-Dateien und Exporten (z. B. CSV aus Fachanwendungen), Datenbereinigung vor einem Import, Berichte und Statistiken, Transformationen von API-Antworten (JSON) sowie Validierungsregeln, die man als kleine, pure Funktionen kombinieren kann.